Indexovanie

Vyhľadávanie informácií

Martin Šeleng

Ukážka problému v information retrievale

- Predpokladajme, že chcete zistiť ktoré Shakesperove diela obsahujú mená Brutus a Ceasar a nie meno Calpurnia.
- Najhlúpejšie riešenie je prečítať všetky Shakespearove diela a zistiť v ktorých je meno Brutus a Ceasar a nie je Calpurnia.
- Pre počítač to vlastne znamená urobiť lineárne preskenovanie dokumentov – *grepping* (podobné ako unix-ovský command *grep*)
- Táto kolekcia obsahuje takmer 1 000 000 slov a pre dnešné počítače to nie je nič moc.

Ale z veľa dôvodov predsa len potrebujeme

1. Spracovať veľké dokumenty veľmi rýchlo. Množstvo dát rastie asi tak rýchlo ako rýchlosť počítačov
2. Umožniť oveľa špecifickejšie operácie (nie len AND, OR, NOT) ale napr. aj NEAR (môže byť reprezentované ako v rámci 5 slov alebo v jednej vete, a pod.)
3. Umožniť usporiadanie získaných dokumentov – nielen získať dokumenty s danými slovami ale aj ich správne usporiadať

Ako vylepšiť lineárne skenovanie dokumentov – Booleovský model

- Cesta ako zrušiť lineárne prehľadávanie dokumentov pre každú query (požiadavku) je vytvorenie indexu jednotlivých dokumentov
- Predstavme si incidence matrix (výskytovú maticu). Pre každý dokument (v našom prípade Shakespearovu hru) či obsahuje určité slovo so všetkých slov, ktoré Shakespeare použil (okolo 32 000 slov)
- Bag-of-Words model
  - **Jano má šedú mačku.**
  - **Šedý Jano má mačku.**
  - **Používaný v NLP a IR**

Booleovský model

- Výsledkom tejto matice je binárna term-document (výraz-dokument). O terme uvažujeme zatiaľ ako o slove

	Anthony and Cleopatra	Julius Ceassar	The Tempest	Hamlet	Othello	Macbeth	...
Anthony	1	1	0	0	0	1	
Brutus	1	1	0	1	0	0	
Caesar	1	1	0	1	1	1	
Calpurnia	0	1	0	0	0	0	
Cleopatra	1	0	0	0	0	0	
mercy	1	0	1	1	1	1	
worser	1	0	1	1	1	0	
...							

Booleovský model – pokračovanie príkladu

- Teda na otázku obsahuje Brutus AND Ceasar AND NOT Calpurnia zoberieme riadkové vektory a vykonáme na nich booleovské operácie
- 1100100 AND 110111 AND NOT 010000=
- 100100 a teda odpoveď na našu otázku sú dokumenty (hry) Anthony and Cleopatra a Hamlet.

Booleovský model

- Definícia: Booleovský model je model v ktorom sa môžu používať query (dopyty) ktoré sú vo forme booleaovskej algebry (kombinácie operátorov AND, OR a NOT).

Realisitickejšie scenário

- Predstavme si že, máme 1 000 000 dokumentov (dokumentom rozumieme napr.: hru, knihu, kapitolu v knihe, web page a pod.)
- Grupu dokumentov budeme označovať ako collection (kolekciu). Niekedy sa to označuje ako corpus (korpus – veľký súbor textových materiálov)
- Predstavme si že každý dokument má okolo 1000 slov (2-3 stránky v knihe)
- Ak uvažujeme že na jeden term (slovo) potrebujeme 6 bajtov tak veľkosť tejto kolekcie je okolo 6GB. V týchto dokumentoch môže byť až 500 000 rôznych termov.
- Vo väčšine prípadov nechceme len vrátiť zoznam dokumentov, ktoré obsahujú dané slová ale vrátiť zoznam relevantných dokumentov (neskôr sa tomuto povenujeme)
- V tomto prípade nemôžeme vytvoriť incidence matrix (výskytovú maticu) pretože by sme potrebovali maticu o veľkosti 500K*1M ktorá má okolo pol trilióna 0 a 1.

Úvaha

- Keď sa však pozrieme na takúto maticu zistíme, že táto matica obsahuje veľmi málo 1 (maximálne 0,2%). Každý dokument má 1 000 slov teda maximálne 1 miliardu rôznych termov.
- Teda je oveľa lepšie pamätať si iba miesta kde sa nachádzajú 1-ky a nie 0-ly

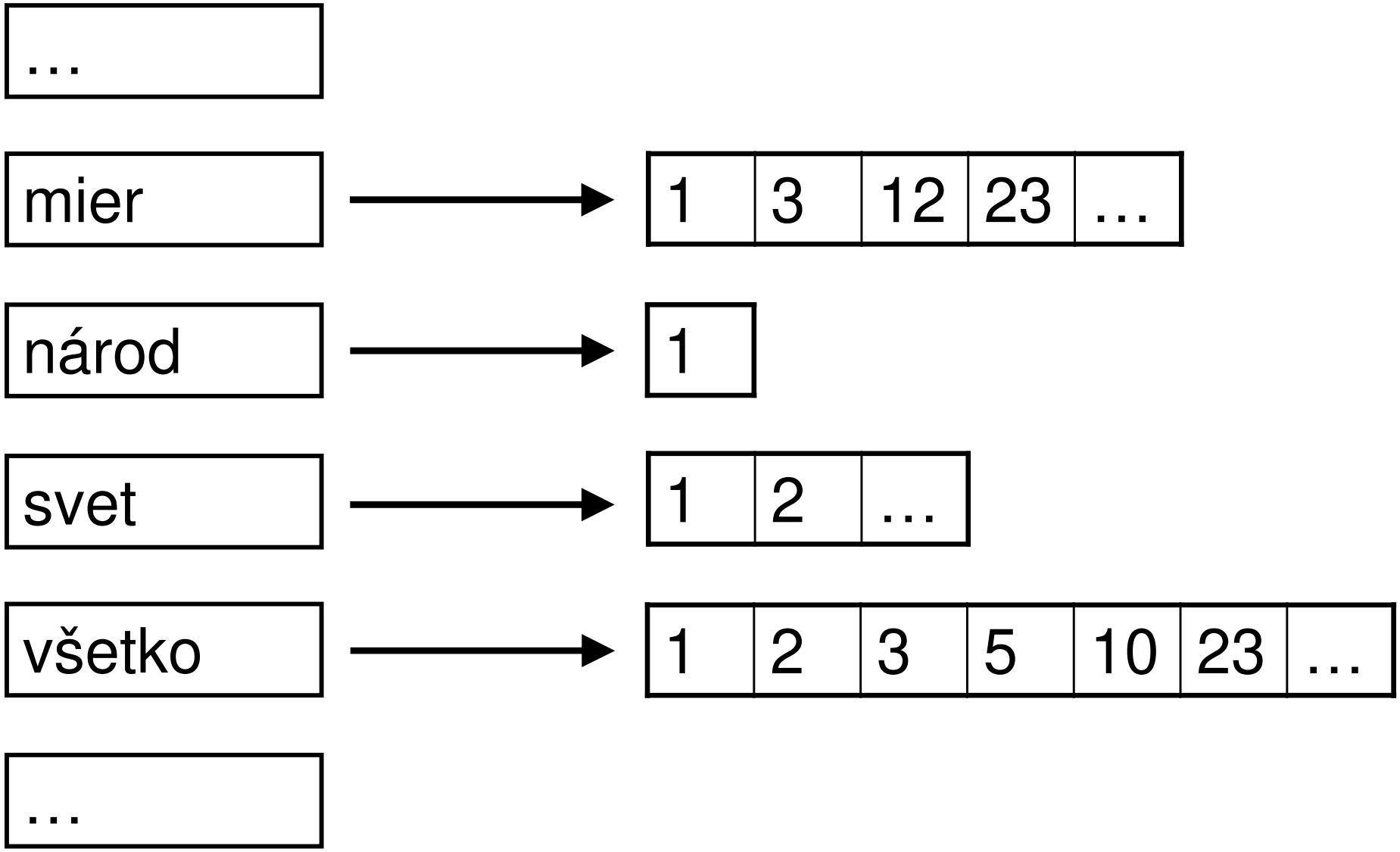
Invertovaný index (Inverted index-Inverted file)

- Dictionary of terms – zoznam všetkých termov držíme v pamäti (**usporiadaný abecedne**)
- Následne pre každý term vytvoríme vektor (**list**) v ktorom je zapísané v ktorom dokumente sa daný term nachádza (neskôr aj pozícia v dokumente) – uložený na disku
- Položka vo vektore (liste) sa volá postings (niečo ako pointer) a celý list (vektor) tzv. **postings list** (**inverted list**)

Ukážka vytvorenie posting listu

- Získať kolekciu dokumentov. „Mier všetkým národom sveta.“
- Tokenizovať text. Mier, všetkým, národom, sveta.
- Spraviť jazykovú analýzu: (mier ako mieriť, miera, peace), všetko, národ, svet
- Abecedne usporiadať a zistiť všetky dokumenty v ktorých sa tieto termy nachádzajú

Ukážka vytvorenie posting listu



Príklad na posting list

- Predkladám dielo o mladíkovi, ktorému som dal meno René, keďže šťastlivo prežil mnoho príhod, na svojich cestách počul a videl veľa poučného, a preto sa napokon i sám cítil ako znovuzrodený.

Jozef Ignác Bajza, Príhody a skúsenosti mládenca Renéhо diel 1.

- Uberali sa rovno do krajiny, patriacej pod moc čierneho orla, prešli už hodný kus cesty a zastali v istom meste ešte s talianskym obyvateľstvom, kde René skúsil veľa.

Jozef Ignác Bajza, Príhody a skúsenosti mládenca Renéhо diel 2.

Pokračovanie príkladu na Posting list

1. Každý dokument má svoje docID
2. Tokenizovať dokumenty a získať základné tvary slov
3. Abecedne usporiadať a priradiť každému termu docID dokumentu v ktorom sa nachádza
4. Vytvoriť posting list

predkladat	1	uberat	2
dielo	1	sa	2
o	1	rovny	2
mladik	1	do	2
ktory	1	krajina	2
ja	1	patrit	2
dat	1	pod	2
meno	1	moc	2
rene	1	cierny	2
kedze	1	orol	2
stastny	1	prejst	2
prezit	1	uz	2
vela	1	vela	2
na	1	kus	2
svoje	1	cesta	2
cesta	1	a	2
pocut	1	zastat	2
a	1	v	2
vidiet	1	isty	2
vela	1	mesto	2
poucny	1	este	2
a	1	s	2
preto	1	taliansky	2
sa	1	obyvatel	2
napokon	1	kde	2
a	1	rene	2
sam	1	skusit	2
citit	1	vela	2
ako	1		
znovu	1		
zrodit	1		

a	1	pocut	1
a	1	pod	2
a	1	poucny	1
a	2	predkladat	1
ako	1	prejst	2
cesta	1	preto	1
cesta	2	prezit	1
citit	1	rene	1
cierny	2	rene	2
dat	1	rovny	2
dielo	1	s	2
do	2	sa	1
este	2	sa	2
isty	2	sam	1
ja	1	skusit	2
kde	2	svoje	1
kedze	1	stastny	1
krajina	2	taliansky	2
ktory	1	uberat	2
kus	2	uz	2
meno	1	v	2
mesto	2	vela	1
mladik	1	vela	1
moc	2	vela	2
na	1	vela	2
napokon	1	vidiet	1
o	1	zastat	2
obyvatel	2	znovu	1
orol	2	zrodit	1
patrit	2		

term	freq	docID/freq					term	freq	docID/freq			
a	4	**1**	3	**2**	1		patrit	1	**2**	1		
ako	1	**1**	1				pocut	1	**1**	1		
cesta	2	**1**	1	**2**	1		pod	1	**2**	1		
citit	1	**1**	1				poucny	1	**1**	1		
cierny	1	**2**	1				predkladat	1	**1**	1		
dat	1	**1**	1				prejst	1	**2**	1		
dielo	1	**1**	1				preto	1	**1**	1		
do	1	**2**	1				prezit	1	**1**	1		
este	1	**2**	1				rene	2	**1**	1	**2**	1
isty	1	**2**	1				rovny	1	**2**			
ja	1	**1**	1				s	1	**2**			
kde	1	**2**	1				sa	2	**1**	1	**2**	1
kedze	1	**1**	1				sam	1	**1**	1		
krajina	1	**2**	1				skusit	1	**2**	1		
ktory	1	**1**	1				svoje	1	**1**	1		
kus	1	**2**	1				stastny	1	**1**	1		
meno	1	**1**	1				taliansky	1	**2**	1		
mesto	1	**2**	1				uberat	1	**2**	1		
mladik	1	**1**	1				uz	1	**2**	1		
moc	1	**2**	1				v	1	**2**	1		
na	1	**1**	1				vela	4	**1**	2	**2**	2
napokon	1	**1**	1				vidiet	1	**1**	1		
o	1	**1**	1				zastat	1	**2**	1		
obyvatel	1	**2**	1				znovu	1	**1**	1		
orol	1	**2**	1				zrodit	1	**1**	1		

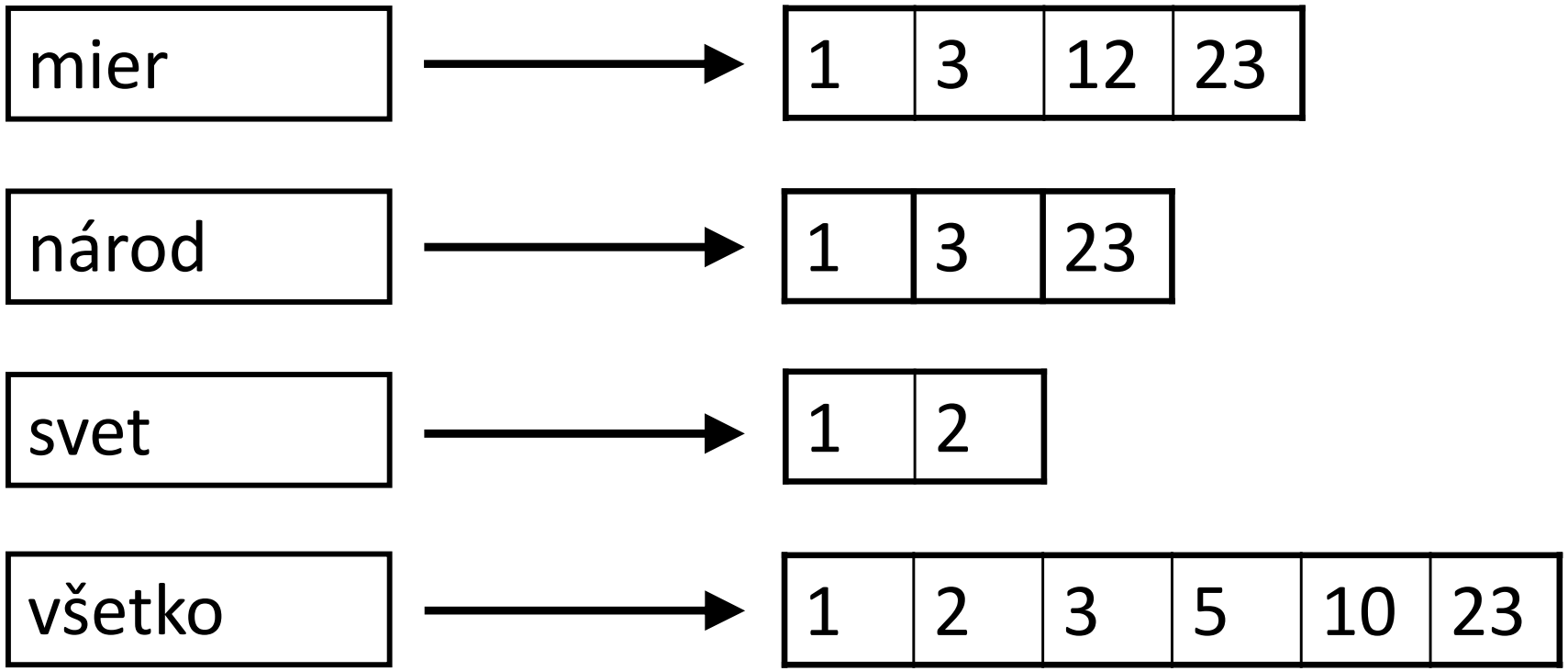
Spracovanie dopytov v booleovskom modeli

- rovná AND cesta = rovny AND cesta
- Nájdi rovny v slovníku
- Získaj posting list
- Nájdi cesta v slovníku
- Získaj posting list
- Urob prienik
- 2 AND 12 = 2
- Najzávažnejšia je operácia prieniku

Spracovanie dopytov v booleovskom modeli – algoritmus prieniku (merging lists) lineárna zložitosť

```
INTERSECT(p1,p2)
answer=[]
while p1<>null and p2<>null
  do if docID(p1)=docID(p2)
        then
              ADD(answer,docID(p1))
              p1=next(p1)
              p2=next(p2)
        else if docID(p1)<docID(p2)
              then p1=next(p1)
              else p2=next(p2)
return answer
```

Ako spracovávať komplikovanejšie dopyty



mier AND národ AND svet

Najvýhodnejší spôsob je usporiadať posting listy podľa veľkosti a takýmto spôsobom vykonáme najmenší počet porovnaní

teda: (svet AND národ) AND mier

Generickejší algoritmus prieniku n posting listov

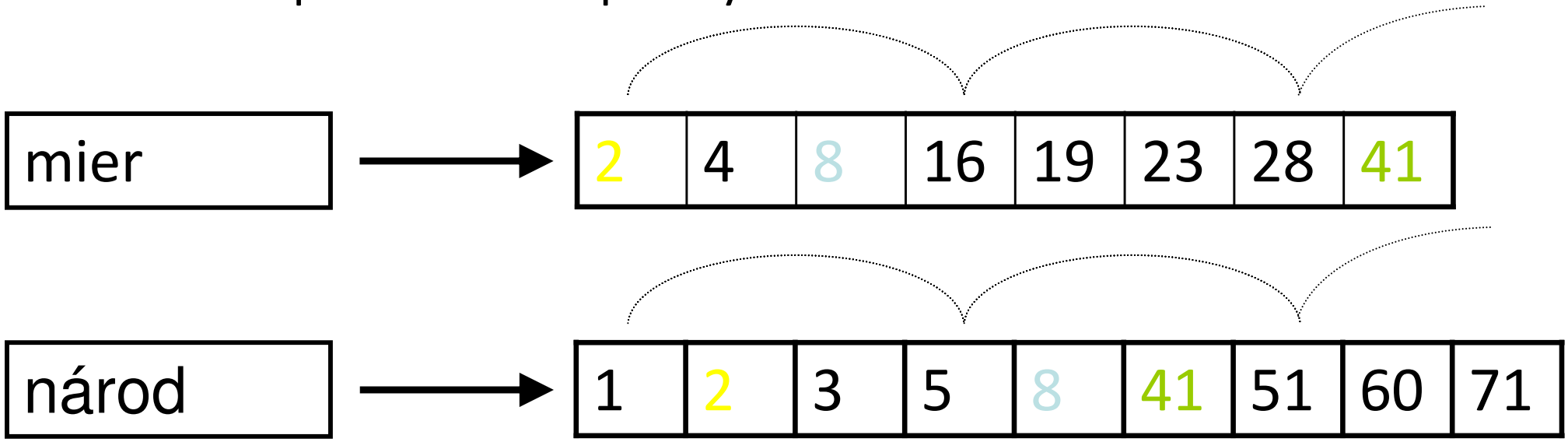
```
INTERSECT(p1,p2,...,pn)
terms=SORTBYINCREASINGFREQUENCY(p1,p2,…,pn)
result=POSTINGS(FIRST(terms))
terms=REST(terms)
while terms<>null and result<>null
  do result=INTERSECT(result,POSTINGS(FIRST(terms)))
     terms=REST(terms)
return result
```

Proximity queries (dokončiť GOOGLE)

- /s /p /k (už asi tiež nie sú – AROUND)
- “ “ exact queries (fráza)
- medzera znamená AND
- AND, |=OR
- word1-word2 => word1word2 OR word1-word2 OR (word1 AND word2)
- word! (word, words, ….)
- veľa ľudí ale stále používa klasické booleovské operátory AND OR NOT
- +word (už Google zrušil) – nepoužívať synonymá, je to to isté ako “word”
- https://ahrefs.com/blog/google-advanced-search-operators/
- https://support.google.com/websearch/answer/2466433?hl=en

Posting list revisited

- Rýchlejší prienik posting listov: Skip pointers, počas tvorenia indexu pridávame skip linky



Nevýhoda pre čiastkové výsledky žiadne skip pointre nemáme
Samozrejme skip pointre pomáhajú len pri AND nie pri OR
Otázkou je ako ďaleko robiť skoky? Empirické riešenie hovorí že, ak máme posting list dĺžky p, skip pointre treba mať dĺžky √p

Algoritmus Intersect with skips

```
INTERSECTWITHSKIPS(p1,p2)
answer=[]
while p1<>null and p2<>null
  do if docID(p1)=docID(p2)
        then
                ADD(answer,docID(p1))
                p1=next(p1)
                p2=next(p2)
        else if docID(p1)<docID(p2)
                then while hasSkip(p1) and
                (docID(skip(p1))<docID(p2))
                        do p1=skip(p1)
                p1=next(p1)
                else while hasSkip(p2) and
                (docID(skip(p2))<docID(p1))
                        do p2=skip(p2)
                p2=next(p2)
return answer
```

Vyhľadávanie fráz „“

- 10% dopytov na webe je fráz
- Na to aby sme podporovali vyhľadávanie fráz už posting listy nemôžu byť iba o tom v ktorom dokumente sa daný term nachádza
- Biword (dvojslovné) indexy
- Positional (pozičné) indexy – častejšie používané

Biword (dvojslovné) indexy

- Mier ľuďom celého sveta
- mier ľudia
- ľudia celý
- celý svet
- Ak by sme chceli vyhľadať viacslovnú frázu tak ju rozbijeme na dvojslovné
- Nevýhoda – nie je zaručené že to čo takýto systém vráti je naozaj to čo používateľ chcel (treba to skontrolovať)
- Existujú indexy kde je veľkosť (čo do počtu slov) pridaných do termov variabilná (phrase index – space problem!!!)

Positional indexes (Pozičné indexy)

- Budeme si pamätať pozíciu termu v každom dokumente a takisto aj počet termov v dokumente

a, 121000: (1,6:(4,10,15,20,29,100);
(2,3:(8,10,34);
(3,1:(1);
(4,2:(34,56);…)

nie, 45002: (1,2:(5,32);
(3,1:(7);
(4,2:(35,57);…)

Positional indexes (Pozičné indexy)

- Ak chceme vyhľadať napr. frázu „a nie“ postupujeme podobne ako v prípade bez pozičných indexov zoberieme term s najmenšou frekvenciou ale pri prieniku musíme sledovať aj pozíciu termov
- 1. dokument pozícia 4 pre a
- 3. dokument je síce prekryv, ale nesedia pozície
- 4. dokument pozície 34 a 56 pre a ...

Algoritmus Intersect with positional indexes

```
POSITIONALINTERSECT(p1,p2,k)
answer=[]
while p1<>null and p2<>null
  do if docID(p1)=docID(p2)
    then l=[]
      pp1=positions(p1)
      pp2=positions(p2)
      while pp1<>null
      do while pp2<>null
        do if abs(pos(pp1)-pos(pp2))≤k
          then ADD(l,pos(pp2))
          else if pos(pp2)>pos(pp1)
            then break
          pp2=next(pp2)
        while l<>[] and abs(l[0]-pos(pp1))>k
          do delete(l[0])
        for each ps € l
          do ADD(answer,(docID(p1),pos(pp1),ps))
        pp1=next(pp1)
      p1=next(p1)
      p2=next(p2)
    else if docID(p1)<docID(p2)
      then p1=next(p1)
      else p2=next(p2)
return answer
```

Konštrukcia indexu

- Čo musíme riešiť
  - **Dokumenty kódované v rôznych formátoch (arabčina, čínština, japončina, ...)**
  - **Súbory, ktoré chceme indexovať nie sú väčšinou lokálne (web crawl)**
  - **Vo firmách sa väčšinou dáta nachádzajú v rôznych „content management“ systémoch**
  - **Index je vo väčšine prípadov komprimovaný (z dôvodu prenosu), teda treba riešiť komprimovanie a dekomprimovanie**

Hardvérové vlastnosti, požiadavky a obmedzenia

- Prístup do pamäte je oveľa rýchlejší ako na disk ($5x10^{-9}$s ku $8x10^{-8}$s na načítanie 1 bajtu) – hovoríme o klasických rotačných diskoch
  - **Riešenie - caching**
- Disk seek time je v priemer 5ms (žiadane dáta ešte neboli načítané)
  - **Minimalizovať načítaciu a transferovaciu dobu súbory ukladať kontinuálne**
  - **Čas na prenos 1 bajtu je približne ($2x10^{-8}$s) u štandardného počítača. Preniesť 10MB v jednom bloku zaberie 0,2+0,005=0,205 sekundy avšak preniesť 100MB ak sú uložené ako 1MB bloky zaberie 0,2+100x0,005=0,7**
  - **Počítaču trvá načítanie 1 bajtu, takisto dlho ako načítanie 1 bloku (8, 16, 32 a 64KB) – buffer**
  - **V IR systémoch majú servery rádovo 10GB pamäte a niekoľko TB diskového priestoru**

Zopakovanie vytvorenia indexu

- Prebehnutie celej kolekcie – vytvorenie párov term-docID
- Usporiadanie podľa termov ako primárneho indexu a docID ako sekundárneho
- Vytvorenie posting listov
- Spočítanie term a dokument frekvencie
- Pre malé kolekcie toto všetko vieme urobiť v pamäti, pre veľké potrebujeme sekundárne úložisko
- Miesto term budeme používať termID

Blocked sort-based indexing pri nedostatku miesta v pamäti

```
BSBINDEXCONSTRUCTION()
  n = 0
  while (všetky dokumenty nie sú spracované)
  do n = n + 1
    block = PARSENEXTBLOCK()
    BSBI-INVERT(block)
    WRITEBLOCKTODISK(block, f_n)
MERGEBLOCKS(f_1, ... f_n; f_Merged)
```

Blocked sort-based indexing pri nedostatku miesta v pamäti

- `block = PARSENEXTBLOCK()`
  - **Páry termID-docID sú načítavané do pamäte (do vopred danej veľkosti – blocku)**
- `BSBI-INVERT(block)`
  - **Urobíme invertovaný posting list**
    - Zoradíme podľa termu
    - K termID pridáme všetky docID
- `WRITEBLOCKTODISK(block, f`$_n$`)`
  - **Zapísať späť na disk**

Single-pass in-memory indexing pri nedostatku miesta v pamäti

- BSBI je síce škálovateľný, ale potrebujeme mapovanie medzi term a termID (pre veľké kolekcie nie je možné držať toto mapovanie v pamäti)
- SPIMI používa termy narozdiel od termID a vie indexovať ľubovoľne veľké kolekcie dokumentov – pokiaľ je dosť diskového priestoru

Single-pass in-memory indexing pri nedostatku miesta v pamäti

```
SPIMI-INVERT(token_stream)
  output_file = NEWFILE()
  dictionary = NEWHASH()
  while (voľná pamäť)
  do token = next(token_stream)
    if term(token) not in dictionary
      then postings_list = ADDTODICTIONARY(dictionary,
  term(token))
      else postings_list = GETPOSTINGSLIST(dictionary,
  term(token))
    if full(postings_list)
      then postings_list = DOUBLEPOSTINGSLIST(dictionary,
  term(token))
    ADDTOPOSTINGSLIST(postings_list, docID(token))
  sorted_terms = SORTTERMS(dictionary)
  WRITEBLOCKTODISK(sorted_terms, dictionary, output_file)
return output_file
```

Single-pass in-memory indexing pri nedostatku miesta v pamäti

- *token_stream* obsahuje páry term-docID
- SPIMI-INVERT je volané dovtedy, kým neprejdeme celú kolekciu
- *token* (riadok 4) je spracovávaný jeden po druhom
- Ak sa term objaví po prvý krát je pridaný do dictionary (ako hash code) a vytvorí sa pre neho posting list (riadok 6)
- Ak už spracovávaný term je v dictionary, tak pre neho získame jeho čiastočný posting list (riadok 7)
- Rozdiel medzi BSBI a SPIMI je v tom, že SPIMI pridá docID priamo do posting listu daného termu (riadok 10), pričom BSBI najprv zozbiera všetky páry termID-docID a potom ich zoradí.
- Posting list má dynamickú veľkosť, ktorá je zväčšovaná (dvojnásobne) podľa potreby (riadok 9)
  - **Určitá nevýhoda voči BSBI, keďže nie celá veľkosť alokovaného miesta v pamäti pre posting list musí byť využitá, avšak stále menšia spotreba pamäte ako pri BSBI**
- SPIMI oproti BSBI má tri výhody:
  - **Je rýchlejší, pretože netreba zoraďovať docID v posting listoch**
  - **Šetrí pamäť, pretože si nepotrebujeme udržiavať info o mapovaní medzi term a termID**
  - **Je možné použiť kompresiu**
- Keď je voľná pamäť vyčerpaná, usporiadame posting listy podľa abecedy (riadok 11) – podľa termu
- Následne uložíme na disk block dát pozostávajúci z dictionary a posting listov (riadok 12)
- Posledným krokom je spojenie jednotlivých výstupov z SPIMI-INVERT (nie je v kóde)

Distribuované indexovanie MapReduce

- TODO – prednáška o MapReduce architektúre

Frekvencia termov a váženie (1)

- každému termu v dokumente môžeme priradiť váhu podľa nasledujúcej definície:
- $tf_{t,d}$ = počet termov *t* v dokumente *d*
  - **tomuto váženiu sa hovorí term frequency**
  - **niekedy – prirodzené váženie**
  - **najjednoduchší spôsob**
- miera zhody (overlap score) je miera podobnosti dokumentu *d* definovaná ako suma cez všetky termy dopytu *q* na počet výskytov termu z dopytu *q* v dokumente *d*.
  - $Score(q,d) = \sum_{t \in q} tf_{t,d}$

Frekvencia termov a váženie (2)

- *tf* má však jednu podstatnú nevýhodu: každému termu priraďuje rovnakú silu
  - **ak si spomenieme na tokenizáciu textu (stopword) a teda napr spojka „a“ nemôže mať takú istú významovú silu ako napr. ľubovoľné podstatné meno**
- takisto *tf* favorizuje dokumenty s väčšou dĺžkou, keďže obsahujú viacej termov
- *tf* vôbec nerieši dĺžku dopytu

Inverse document frequency (1)

- Jedným z riešení je zaviesť ďalšiu vážiacu premennú: cf_t (*collection frequency*) ktorá vlastne vyjadruje počet výskytu termu *t* v celej kolekcii dokumentov. A idea by bola v tom že znížime váhu termu (*tf*) podľa počtu výskytov termu v celej kolekcii
- Je výhodnejšie ale použiť inú vážiacu premennú df_t (*document frequency*), ktorá vyjadruje v koľkých dokumentoch sa nachádza term *t*
- Dôvod prečo treba uprednostniť *df* pred *cf* je tento:
  - **Predstavme si kolekciu dokumentov o poistení (nevyhadzujeme stop slová) a ešte iných dokumentoch, kde sa poistení vôbec nehovorí a dostaneme túto tabuľku:**
- Takže opäť sme na začiatku, ani jedna zo štatistík, sama o sebe nie je dosť výpovedná

Slovo	cf	df
nie	10023	8022
poistenie	10134	4507

Inverse document frequency (2)

- Ako môžeme premennú *df* použiť na váženie?
- Definícia: Inverse document frequency:

$$idf_t = \log \frac{N}{df_t}$$

  - ***N* je počet dokumentov, základ môže byť hocijaký (štandardne 10)**
- Čím je df_t menšie tým je idf_t významnejšie

Inverse document frequency (3)

- $tf - idf_{t,d} = tf_{t,d} \times idf_t$
- *tf-idf*$_t$ priraďuje termu *t* váhu:
  - **Najvyššiu ak sa term *t* vyskytuje veľakrát v malom počte dokumentov**
  - **Nižšiu ak sa term *t* vyskytuje menej krát v dokumente alebo sa vyskytuje vo veľa dokumentoch**
  - **Najnižšiu ak sa term *t* vyskytuje vo veľa dokumentoch (teoreticky vo všetkých)**

Dokument je vlastne vektor

- V tomto bode sa môžeme na dokument pozrieť ako na vektor kde hodnoty tohto vektora sú vlastne váhy jednotlivých termov.
  - **Ak sa term v dokumente nenachádza jeho váha je 0.**
- Miera zhody (Overlap score)
  - $\boldsymbol{Score(q,d) = \sum_{t \in q} tf - idf_{t,d}}$

Modifikácie *tf-idf*$_t$ – logaritmické škálovanie (weighted frequency) (1)

- Je otázne či to že term *t* sa v dokumente *d* vyskytuje povedzme 20x má mať naozaj 20x väčšiu silu ako keď sa tam vyskytuje len raz?
- Najčastejšia modifikácia je že sa použije logaritmus (upraví sa priebeh – weighted frequency):

$$w_{t,f} = \begin{cases} 1 + \log(tf_{t,d}) & tf_{t,d} > 0 \\ 0 & tf_{t,d} \leq 0 \end{cases}$$

- $wf - idf_{t,d} = wf_{t,d} \times idf_t$

Modifikácie *tf-idf*$_t$ – Normalizácia maximálnym *tf* (2)

$$ntf_{t,d} = \alpha + (1-\alpha)\frac{tf_{t,d}}{tf_{max}(d)}, tf_{max}(d) = \max_{x \in d} tf_{x,d}$$

α – je tzv. škálovací koeficient, štandardne nastavený na 0,5

Táto normalizácia upravuje nasledovné anomálie:

- ak sa v dokumente vyskytuje často slovo ktoré nemá s daným dokumentom nič spoločné
- ak sa vyskytuje v dokumente príliš často slovo a prebíja ostatné slová (rozdelenie je viac skreslené)
- okopírujeme celý dokument 2x (to isté) – toto sa používalo na oblbnutie vyhľadávacích strojov ako: Google, Yahoo, Metacrawler

Nerieši však toto:

- zmena v slovníku stop slov mení dosť významne váhy

Modifikácie *tf-idf*$_t$ –Dĺžka dokumentu a Euklidovská normalizácia (3)

- predchádzajúce úvahy nebrali na zreteľ dĺžku dokumentu (to že je term príliš často v nejakom dokumente môže byť spôsobené a aj je určite i jeho dĺžkou, resp. dlhšie dokumenty obsahujú viac rôznych slov)
- Definícia: Majme váhy w_1, w_2,..., w_n termov v dokumente. Tieto môžu byť či *tf*, *tf-idf* alebo modifikácia *tf-idf* definuje nový set

$$\frac{w_1}{\sqrt{w_1^2 + w_2^2 + \cdots + w_n^2}}, \frac{w_2}{\sqrt{w_1^2 + w_2^2 + \cdots + w_n^2}}, \ldots, \frac{w_n}{\sqrt{w_1^2 + w_2^2 + \cdots + w_n^2}}$$

- opäť môžeme zadefinovať overlap score
- výhoda tejto normalizácie je že rieši aj problém s opakovaním celého dokumentu

Modifikácie tf_t – Sumarizácia (4)

- prirodzená $tf_{t,d}$
- logaritmická $1 + \log(tf_{t,d})$
- normalizácia max. *tf* $\alpha + (1 - \alpha)\frac{tf_{t,d}}{tf_{max}(d)}$
- booleaovska $tf_{t,d} > 0 ? 1 : 0$
- logpriemerovaná $\frac{1 + \log(tf_{t,d})}{1 + \log\left(\text{avg}_{t \in d}(tf_{t,d})\right)}$

Modifikácie df_t – Sumarizácia (5)

- prirodzená df_t 1
- idf_t $\log \frac{N}{df_t}$
- probabilistic *idf* $max\left\{0, \log \frac{N - df_t}{df_t}\right\}$

Normalizácie – Sumarizácia (6)

- Žiadna 1
- Euklidovská/Kosínusová $\frac{1}{\sqrt{w_1^2 + w_2^2 + \cdots + w_n^2}}$
- Bajtová (používaná v Okapi BM25) $\frac{1}{DocumentCharLength^{\alpha}}, \alpha < 1$

Príklad na skóre dokumentu (1)

- Score(q,d)=$\sum_{t€q}$ $tf\text{-}idf_{t,d}$
- Score(q,d)=$\sum_{t€q}$ $w_{t,q} \times w_{t,d}$
- Predstavme si dopyt skúška získavanie znalostí a hypotetickú množinu dokumentov
  - **N=10000 dokumentov**
  - **kde frekvencia termov (*df*) skúška, získať, znalosť, predmet sú 50, 500, 1000, 100**
  - **a v dokumentoch ktorým budeme počítame skóre (rank) sa nachádzajú v počtoch 1, 0, 2, 3, resp. 2, 0, 1, 3.**
  - **Určme skóre týchto dokumentov**
    - pre výpočet váhy dopytu použijeme *tf-idf*
    - pre výpočet váhy dokumentu *tf*, pričom použijeme logaritmické škálovanie a normalizáciu pomocou Euklida

Príklad na skóre dokumentu (2)

N	Dopyt					Dokument			10000
term	*tf*	*df*	*idf*	*tf-idf*	$w_{t,q}$	*tf*	*wf(log. š.)*	$w_{t,d}$	*score*
skúška	1	50	2,30103	2,30103	2,30103	1	1	0,452931	1,042208
získať	1	500	1,30103	1,30103	1,30103	0	0	0	0
znalosť	1	1000	1	1	1	2	1,30103	0,589277	0,589277
predmet	0	100	2	0	0	3	1,477121	0,669034	0
						Dĺžka:	**2,207841**	**Skóre:**	**1,631485**
term	*tf*	*df*	*idf*	*tf-idf*	$w_{t,q}$	*tf*	*wf(log. š.)*	$w_{t,d}$	*score*
skúška	1	50	2,30103	2,30103	2,30103	2	1,30103	0,589277	1,355944
získať	1	500	1,30103	1,30103	1,30103	0	0	0	0
znalosť	1	1000	1	1	1	1	1	0,452931	0,452931
predmet	0	100	2	0	0	3	1,477121	0,669034	0
						Dĺžka:	**2,207841**	**Skóre:**	**1,808874**

Vektorový model (1)

- Tak ako sme pred chvíľou definovali dokument ako vektor tak množinu dokumentov môžeme nazvať vektorovým priestorom.
- Tento model je v “information retrieval” základom a používa sa na určenie ranku (skóre) dokumentu na základe dopytu, klasifikácie a zhlukovania.

Vektorový model (2)

- *V(d)* je vektor dokumentu *d* pozostávajúci z váh jednotlivých termov.
- Pre ďalšie úvahy je úplne jedno o ktoré váhy sa jedná (štandardne sa používa *tf-idf*)
- Množina dokumentov potom môže byť reprezentovaná ako vektorový priestor kde každý term tvorí 1os

Vektorový model (3)

- Prečo meníme dokumenty a dopyty na vektory?
  - Opustíme jednoduchší Booleanovský model
  - Vieme vymyslieť metódy vyhľadávania informácií založené na ocenení dokumentov/dopytov
  - Vymyslenie skórovania pre usporiadanie vrátených výsledkov
  - Váženie termov, nám umožňuje počítať skóre podobnosti dokumentov
  - Vektorový priestorový model je čistý matematický model
  - Slovne podobné dokumenty sa budú v priestore nachádzať blízko seba, resp. vektory budú ukazovať podobným smerom

Kosínusova miera – miera podobnosti (1)

- Ako budeme počítať similaritu medzi dvomi dokumentmi (dopyt je vlastne tiež dokument!!!)?
- Prvá úvaha dĺžka vektora.
- Nevýhoda: relatívne rozdelenia termov v dvoch dokumentoch sú rovnaké avšak jeden je podstatne dlhší ako druhý.
- Riešenie:

$$\mathrm{sim}(d_1, d_2) = \frac{V\left(\overrightarrow{d_1}\right) V\left(\overrightarrow{d_2}\right)}{\left|V\left(\overrightarrow{d_1}\right)\right| \left|V\left(\overrightarrow{d_2}\right)\right|} = \cos\left|\sphericalangle V\left(\overrightarrow{d_1}\right) V\left(\overrightarrow{d_2}\right)\right|$$

Kosínusova miera – miera podobnosti (2)

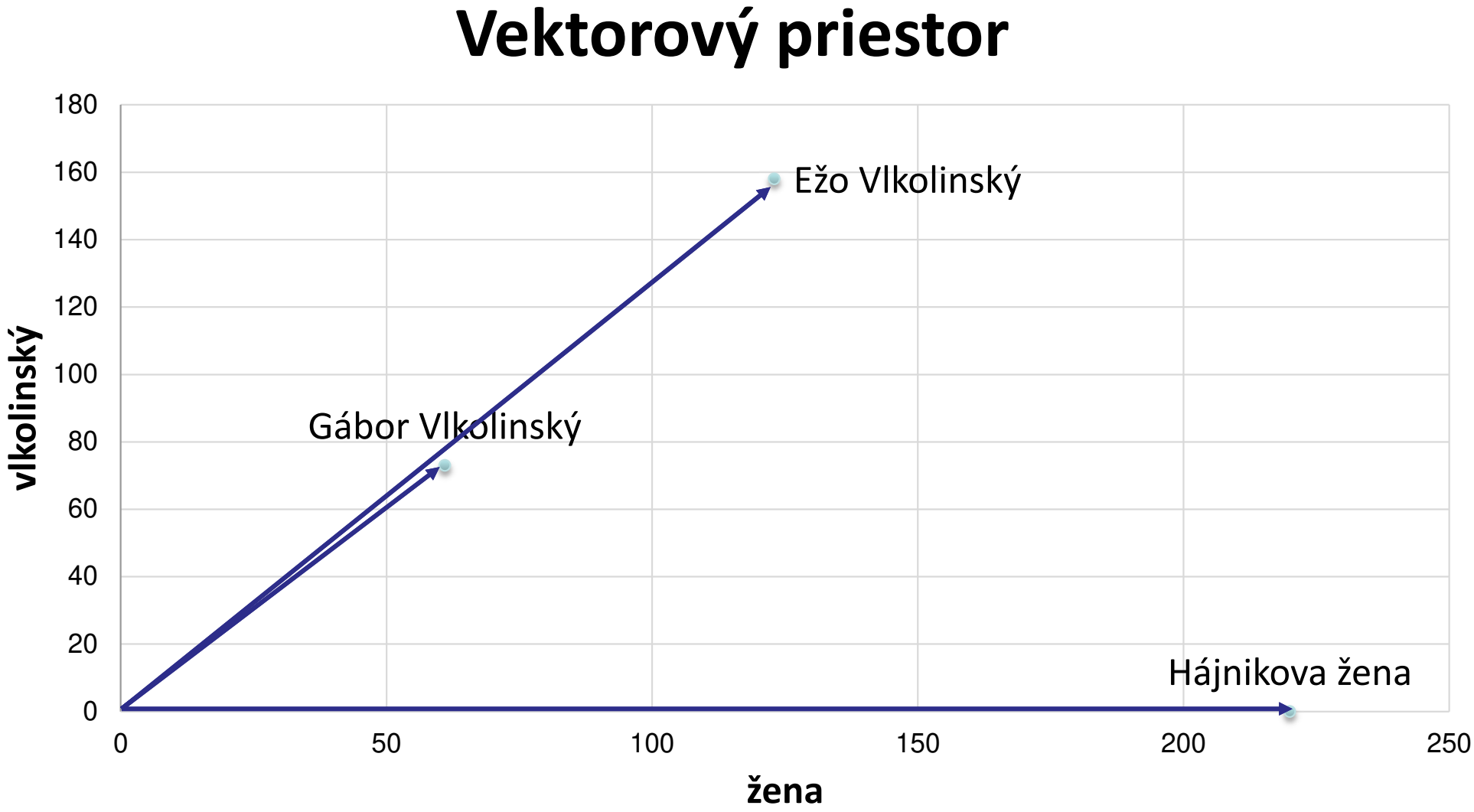
- Dáva nám možnosť použiť symetrické skóre
  - **ak d_1 je blízko (podobné) d_2, tak to platí aj naopak**
- Dáva nám tranzitívnosť
  - **ak d_1 je blízko (podobné) d_2 a d_2 je blízko (podobné) d_3, potom d_1 je blízko (podobné) d_3**
- Žiaden dokument nie je bližšie k dokumentu d_1 ako d_1 samo k sebe
- Ak sú vektory normalizované:
  - **Skóre podobnosti je iba skalárny súčin dvoch vektorov (rýchla operácia)**

Príklad na kosínusovú mieru (1)

- Majme 3 diela Ežo Vlkolinský, Gábor Vlkolinský a Hájnikova Žena v tabuľke je výskyt týchto 3 slov: žena, Vlkolinský, dedina v týchto 3 dielach nasledovná (Euklidovská normalizácia):

term/dielo	Ežo	Gábor	Žena
žena	123	61	220
vlkolinský	158	73	0
dedina	35	21	34

Príklad na kosínusovú mieru (2)



Príklad na kosínusovú mieru (3)

term/dielo	Ežo Vlkolinský (tf)	Gábor Vlkolinský (tf)	Hájnikova žena (tf)
žena	123	61	220
vlkolinský	158	73	0
dedina	35	21	34
term/dielo	Ežo Vlkolinský (euklidovská norm.)	Gábor Vlkolinský (euklidovská norm.)	Hájnikova žena (euklidovská norm.)
žena	0,605111582	0,626143459	0,988267603
vlkolinský	0,777297805	0,749319222	0
dedina	0,172186223	0,215557584	0,152732266
Porovnanie:	**1,2**	**0,998446892**	
	1,3	**0,624310565**	
	2,3	**0,651719894**	

No a načo to bolo celé dobré?

- $\mathrm{sim}(d_1, d_2) = \frac{V(\overrightarrow{d_1})V(\overrightarrow{d_2})}{|V(\overrightarrow{d_1})||V(\overrightarrow{d_2})|} = \cos\left|\sphericalangle V\left(\overrightarrow{d_1}\right) V\left(\overrightarrow{d_2}\right)\right|$
- Predpokladajme teraz že používateľ zadal do vyhľadávača dopyt žena a dedina. Ktorý dokument je tomuto dopytu najbližšie?

Dokončenie príkladu na kosínusovu mieru

term	Dopyt (tf)	Ežo Vlkolinský (tf)	Gábor Vlkolinský (tf)	Hájnikova žena (tf)
žena	1	123	61	220
vlkolinský	0	158	73	0
dedina	1	35	21	34
term	Dopyt (euklidovská norm.)	Ežo Vlkolinský (euklidovská norm.)	Gábor Vlkolinský (euklidovská norm.)	Hájnikova žena (euklidovská norm.)
žena	0,707106781	0,605111582	0,626143459	0,988267603
vlkolinský	0	0,777297805	0,749319222	0
dedina	0,707106781	0,172186223	0,215557584	0,152732266
	dopyt,1	**0,549632549**		
	dopyt,2	**0,595172516**		
	dopyt,3	**0,806808745**		

Podobnosť na základe metrík/korelácií

- Euklidovská vzdialenosť – L2 metrika
- Manhattan podobnosť – L1 metrika
- Snp metrika:
- Sokalova metrika:
- Kosínusová korelácia/metrika:
  - **klasický**
  - **latentný sémantický**
  - **pravdepodobnostný**

$$\text{sim}\left(\overrightarrow{d_i}, \overrightarrow{d_j}\right) = \sqrt{\sum_{k=1}^{n} (w_k - v_k)^2}$$

$$\text{sim}\left(\overrightarrow{d_i}, \overrightarrow{d_j}\right) = \sum_{k=1}^{n} |w_k - v_k|$$

$$\text{sim}\left(\overrightarrow{d_i}, \overrightarrow{d_j}\right) = \max_k |w_k - v_k|$$

$$\text{sim}\left(\overrightarrow{d_i}, \overrightarrow{d_j}\right) = \frac{\sum_{k=1}^{n} \sqrt{(w_k - v_k)^2}}{n}$$

Okapi BM (Best Matching) 25 score (BM25F)

- $score(q,d) = \sum_{i=1}^{n} idf_{q_i} \frac{tf_{q_i,d}(k_1+1)}{tf_{q_i,d}+k_1\left(1-b+b\frac{|d|}{|avgd|}\right)}$

$|d|$ – dĺžka dokumentu, $|avgd|$ – priemerná dĺžka dokumentu

$$k_1 \in \langle 1{,}2; 2\rangle, b = 0{,}75, idf_{q_i} = \log \frac{N - df_{q_i} + 0{,}5}{df_{q_i} + 0.5}$$

- *idf* má jednu nevýhodu:
  - **Ak sa nejaký term nachádza viac než v polovici dokumentov – vychádza mu *idf* záporné!!! To má samozrejme za následok penalizáciu vzhľadom na skóre dokumentu voči inému dokumentu resp. dopytu.**
- riešenie:
  - **Nastaviť minimálne *idf* na 0, aby sme odignorovali zvyčajné slová**
  - **Nastaviť minimálne *idf* na ε, aby sme úplne neodignorovali zvyčajné slová**
  - **Nahradiť *idf* funkciu, takou (podobnou) funkciou, ktorá je striktne pozitívna**

Idf interpretácia, podľa teórie informácií

- Pravdepodobnosť že, náhodne vybratý dokument *d* bude obsahovať term *q* je: $\frac{df_q}{N}$, kde *N* je počet dokumentov,
- teda informácia/entropia že, dokument *d* obsahuje term *q* je: $-\log\frac{df_q}{N} = \log\frac{N}{df_q}$.
- Predpokladajme že, máme 2 nezávislé termy q_1 a q_2 vyskytujúce sa v kolekcii, aká je pravdepodobnosť že, sa obidva vyskytujú v dokumente *d*? $\frac{df_{q_1}}{N} \times \frac{df_{q_2}}{N}$.
- Obsah informácie/entropie ktorá, z toho vyplýva je: $\sum_{i=1}^{2} \log\frac{N}{df_{q_i}}$ a toto je v podstate, to čo hovorí *idf* v predchádzajúcej definícii, s nejakou malou modifikáciou.